

蓝桥杯30天算法冲刺集训



Day-23 无向图的连通分量与 tarjan

连通分量是指一个无向图的一个极大连通子图，连通分量中的任意两个点都存在一条路径可以直接或间接互相到达。

对于一个无向图，如何求出它的所有连通分量呢？

算法一：图上 DFS

对于一个没有访问过的点，我们从这个点开始DFS，访问所有还没有访问过的点，每访问到一个点，我们就将其标记为已访问。在某一个点 DFS 结束时，所有在这一轮访问到的点就在一个连通分量中。显然该算法的时间复杂度为 $O(n + m)$ 。

参考代码：

```
const int N = 1e5+5;
int n, m;
int fa[N], vis[N], root; // fa[i]: 节点 i 所在连通分量的代表元素
int vector<int> g[N];
void dfs(int u) {
    vis[u] = 1;
    fa[u] = root;
    for(auto v : g[u]) {
        if(!vis[v])
            dfs(v);
    }
}

for(int i = 1; i <= n; i++) {
    if(!vis[i]) {
        root = i;
        dfs(i);
    }
}
```

```
}
```

算法二：并查集

通过扫描所有的边，不断合并两个连通分量，最终形成图的各个连通分量。算法时间复杂度为： $O(n + m\alpha(n))$ 。

参考代码：

```
const int N = 1e5+5;
int n, m;
int fa[N];
int find(int x) {
    if(fa[x] == x) return x;
    return fa[x] = find(fa[x]);
}
void merge(int x, int y) {
    x = find(x);
    y = find(y);
    fa[x] = y;
}
int main() {
    cin >> n >> m;
    for(int i = 1; i <= n; i++) fa[i] = i;
    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        merge(u, v);
    }
}
```

无向图的割点和桥

前置概念（定义）

点连通度：在一个无向连通图中，如果有一个顶点集合 V ，删除顶点集合 V 以及与 V 中顶点相连（至少有一端在 V 中）的所有边后，原图不连通，就称这个点集 V 为 **割点集合**。大小为 1 的点割集又被称作 **割点**。

一个图的点连通度：最小割点集合中的顶点数。

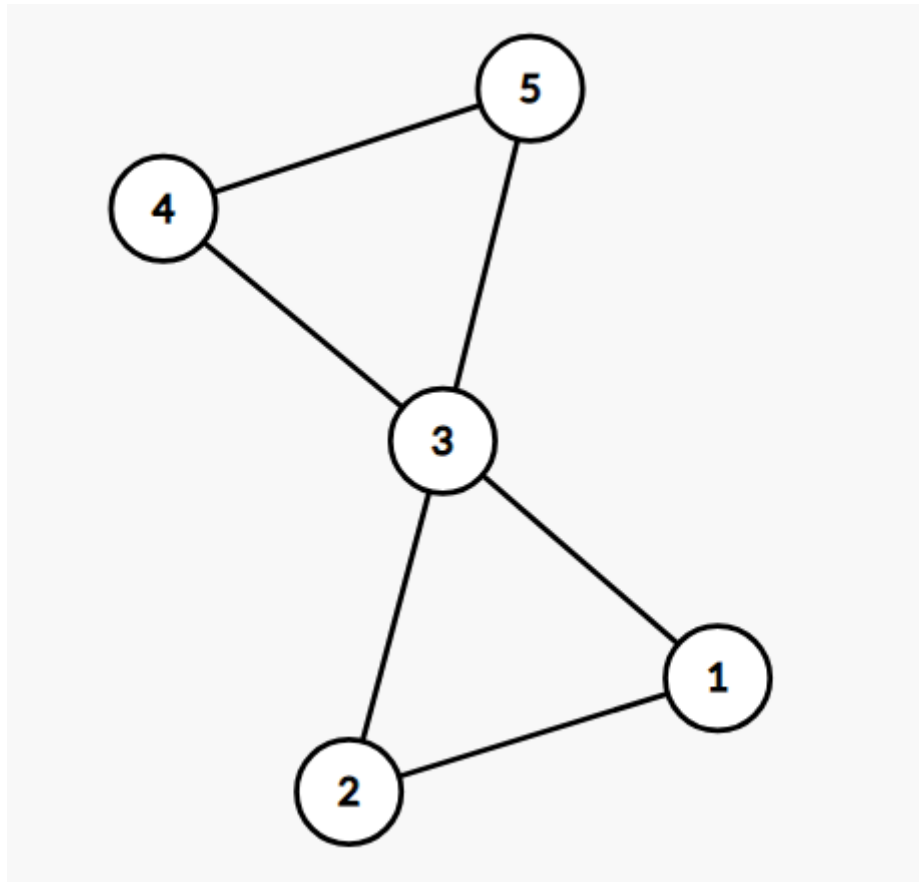
边连通度：在一个无向连通图中，如果有一个边集合 E ，删除这个边集 E 中的边以后，原图不连通，就称这个边集 E 为 **割边集合**。

一个图的边连通度：最小割边集合中的边数。大小为 1 的边割集又被称作 **桥**。

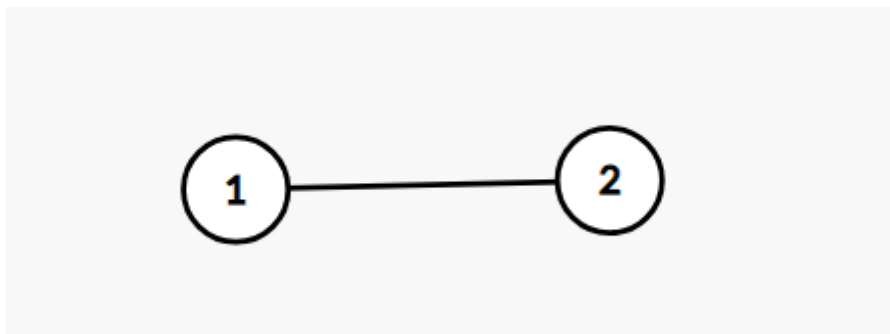
点双连通图：如果一个无向连通图的点连通度大于 1，则称该图是点双连通的（point biconnected），简称点双连通或点重连通（即删去任意一点原图仍然连通）。换句话说，**没有割点的连通图是点双连通的**。

边双连通图：如果一个无向连通图的边连通度大于 1，则称该图是边双连通的（edge biconnected），简称边双连通或边重连通（即删去任意一条边原图仍然连通）。换句话说，**没有桥的连通图是边双连通的**。

割点：一个图有割点，当且仅当这个图的点连通度为 1 时，割点集合的唯一元素被称为割点（cut point）（即删去割点原图不连通）。一个图可能有多个割点。有割点的图不一定有割边，如下图所示，结点 3 为割点，但是图中没有割边。

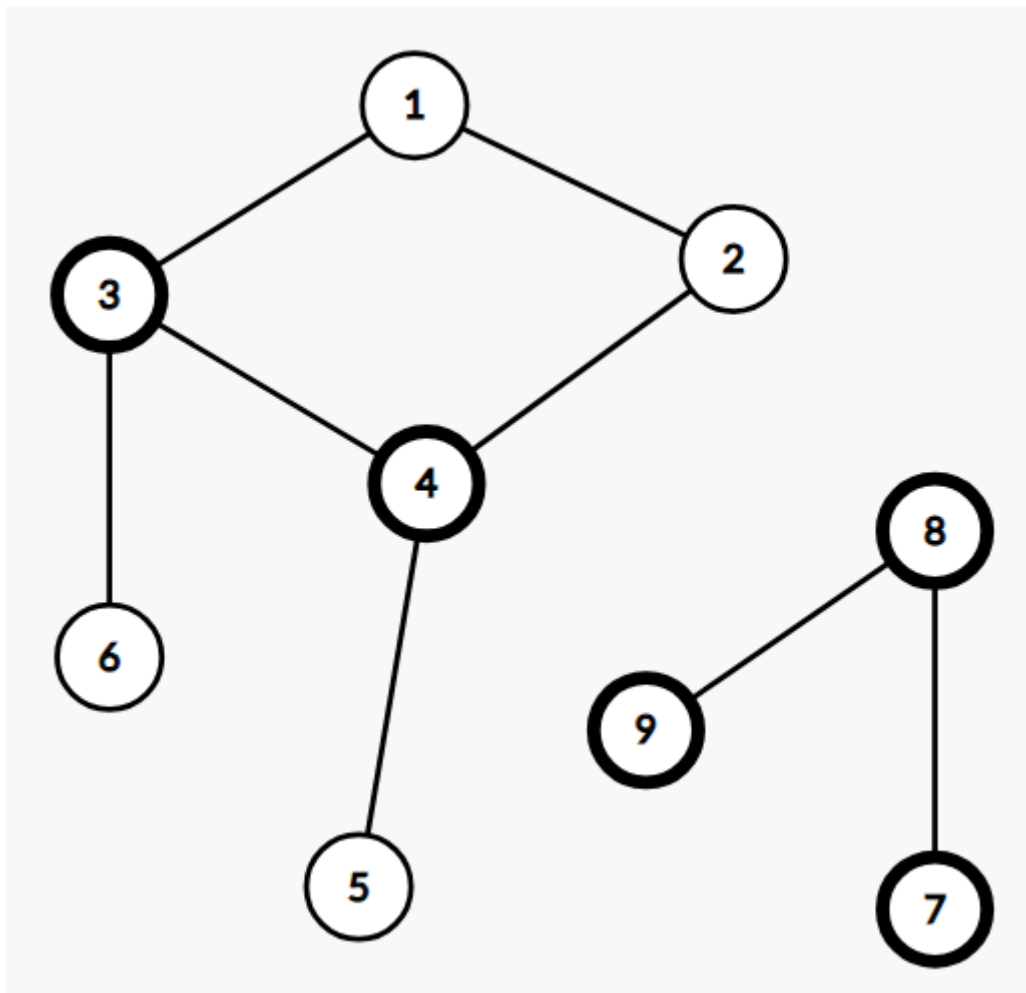


割边（桥）： 如果一个无向连通图的边连通度大于 1，则称该图是边双连通的(edge biconnected)，简称双连通或重连通(即删去任意一边原图仍然连通)。一个图有桥，当且仅当这个图的边连通度为 1 时，割边集合的唯一元素被称为桥(bridge)(即删去桥原图不连通)。一个图可能有多个桥。有割边的图也不一定有割点，如下图所示。



根据定义做两个猜想：两个割点之间的边一定是割边。割边的两个端点一定是割点。

可惜的是，这两个猜想都是错误的！反例如下图：



上图中，结点 3,4 为割点，但是边 (3,4) 不是割边。边 (7,8), (8,9) 为割边，但点 7,9 不是割点。

通俗版定义

对于一个无向图，如果把一个点删除后这个图的连通块（极大连通分量）数增加了，那么这个点就是这个图的割点（又称割顶）。

Tarjan 求无向图的割点和桥

根据著名计算机科学家 Robert Tarjan 的名字命名的 Tarjan 算法能够在线性时间内求出无向图的割点与桥，进一步可求出无向图的双连通分量。在有向图方面，Tarjan 算法能够求出有向图的强连通分量、必经点与必经边(下一节将会介绍)。Robert Tarjan 在数据结构方面也做出了很多卓有成效的工作，包括证明并查集的时间复杂度，提出斐波那契堆、Splay Tree 和 Link-Cut Tree 等。

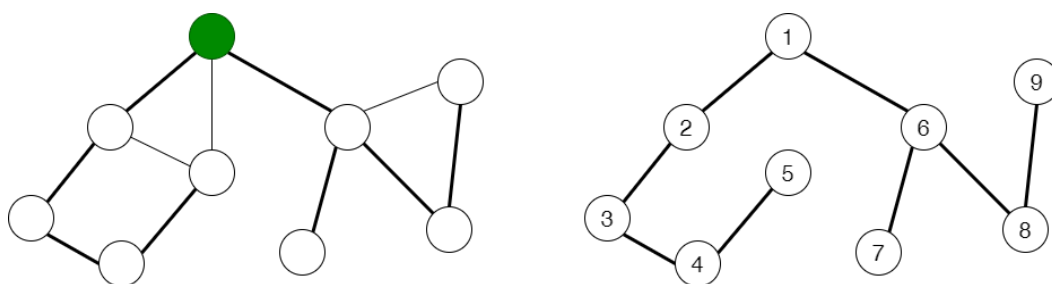
1. 时间戳

在图的 **深度优先遍历** 过程中，按照每个节点第一次被访问的时间顺序，依次给予 N 个节点 $1 \sim N$ 的整数标记，该标记就被称为“时间戳”，记为 $dfn[x]$ 。

2. 搜索树

在无向连通图中任选一个节点出发进行深度优先遍历,每个点只访问一次。所有发生递归的边 (x, y) (换言之，从 x 到 y 是对 y 的第一次访问) 构成一棵树，我们把它称为“无向连通图的搜索树”。当然，一般无向图（不定连通）的各个连通块的搜索树构成无向图的“搜索森林”。

下图左侧展示了一张无向连通图，绿色节点是深度优先遍历的起点，加粗的边是“发生递归”的边（假设我们在遇到多个分支时，总是优先访问最靠左的一条）。右侧展示了深度优先遍历的搜索树，并标注了节点的时间戳。



3. 追溯值

除了时间戳之外，Tarjan 算法还引入了一个“追溯值” $low[x]$ 。设 $subtree(x)$ 表示搜索树中以 x 为根的子树。 $low[x]$ 定义为以下节点的时间戳的最小值：

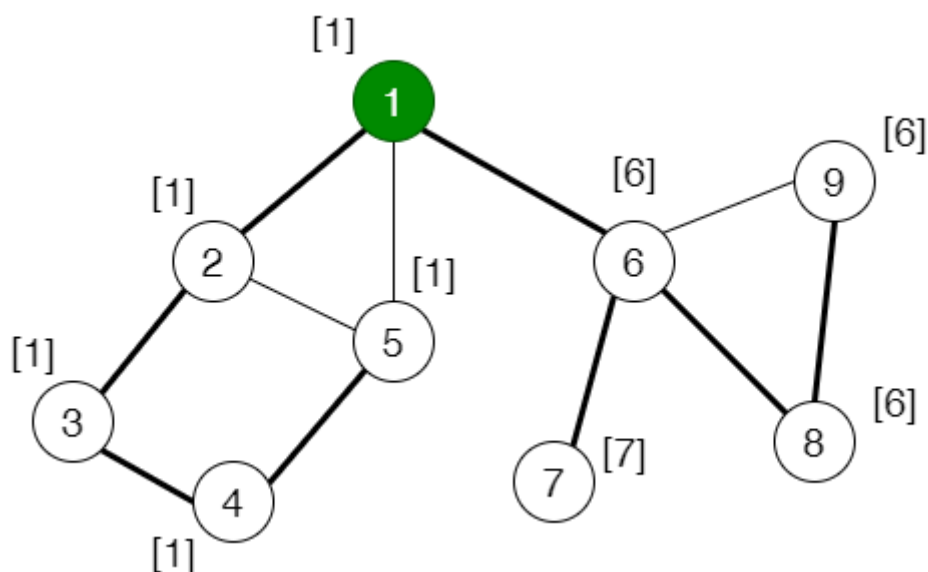
1. $subtree(x)$ 中的节点。
2. 通过一条不在搜索树上的边，能够到达 $subtree(x)$ 的节点。

简而言之， x 的追溯值 $low[x]$ 定义为 x 或 x 的子树能够回溯到的最早的栈中节点的 dfn 值。

以上图为例。为了叙述简便，我们用时间戳代替节点编号。 $subtree(2) = \{2, 3, 4, 5\}$ 。另外，节点 1 通过不在搜索树上的边 $(1, 5)$ 能够到达 $subtree(2)$ 。所以 $low[2] = 1$ 。

根据定义，为了计算 $low[x]$ ，应该先令 $low[x] = dfn[x]$ ，然后考虑从 x 出发的每条边 (x, y) ：

- 若在搜索树上 x 是 y 的父节点，则令 $low[x] = \min(low[x], low[y])$ 。
- 若无向边 (x, y) 不是搜索树上的边，则令 $low[x] = \min(low[x], dfn[y])$ 。下图的中括号里的数值标注了每个节点的“追溯值” low 。



割边判定法则

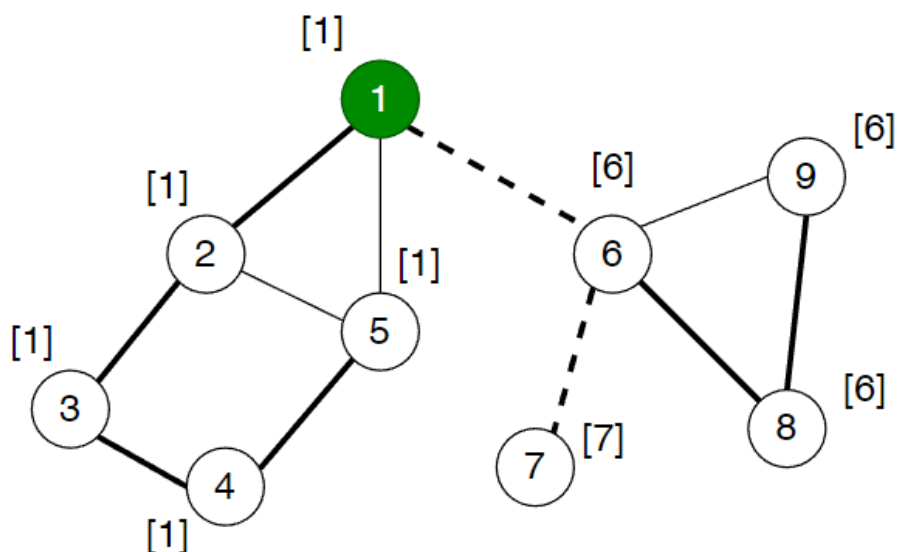
无向边 (x, y) 是桥，当且仅当搜索树上存在 x 的一个子节点 y ，满足：

$$dfn[x] < low[y]$$

根据定义， $dfn[x] < low[y]$ 说明从 $subtree(y)$ 出发在不经过 (x, y) 的前提下，不管走哪条边，都无法到达 x 或比 x 更早访问的节点。若把 (x, y) 删除，则 $subtree(y)$ 就好像形成了一个封闭的环境，与节点 x 没有边相连，图断开成了两部分。因此 (x, y) 是割边。

反之，若不存在这样的子节点 y ，使得 $dfn[x] < low[y]$ ，则说明每个 $subtree(y)$ 都能绕行其他边到达 x 或比 x 更早访问的节点， (x, y) 自然就不是割边。

下面的无向图中有两条“桥”边，用虚线标识。读者不难发现，桥一定是搜索树中的边，并且一个简单环中的边一定都不是桥。



下面的程序求出一张无向图中所有的桥。特别需要注意，因为我们遍历的是无向图，所以从每个点 x 出发，总能访问到它的父节点 fa 。根据 $low[]$ 的计算方法， (x, fa) 属于搜索树上的边(无向边=两条双向边)，且 fa 不是 x 的子节点故不能用 fa 的时间来更新 $low[x]$ 。

但是，如果仅记录每个节点的父节点，会无法处理重边的情况——当 x 与 fa 之间有多条边时， (x, fa) 一定不是桥。在这些重复的边中，只有一条算是“搜索树上的边”，其他的几条都不算。故有重边时， $dfn[fa]$ 能用来更新 $low[x]$ 。

一个好的解决方案是：改为记录“递归进入每个节点的边的编号”。编号可认为是边在邻接表中存储的下标位置。我们可以用“成对变换”技巧（**使用链式前向星存边**）。把无向图的每条边看作双向边，成对存储在下标“2和3”、“4和5”、“6和7”、.....处。若沿着编号为 i 的边递归进入了节点 x ，则忽略从 x 出发的编号为 $i \text{ xor } 1$ 的边，通过其他边计算 $low[x]$ 即可。

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5+5;
int head[N], ver[N*2], Next[N*2];
int dfn[N], low[N], n, m, tot = 1, num;
bool bridge[N*2];

void add(int x, int y) {
    ver[++tot] = y;
    Next[tot] = head[x];
```

```

        head[x] = tot;
    }

    void tarjan(int x, int in_edge) {
        dfn[x] = low[x] = ++num;
        for(int i = head[x]; i; i = Next[i]) {
            int y = ver[i];
            if(!dfn[y]) {
                tarjan(y, i);
                low[x] = min(low[x], low[y]);
                if(low[y] > dfn[x]) { // 是割边 (桥)
                    bridge[i] = bridge[i^1] = true;
                }
            } // 否则, 不是搜索树边, 且不是同一条边
            else if(i != (in_edge^1)) {
                low[x] = min(low[x], dfn[y]);
            }
        }
    }

    int main(){
        cin >> n >> m;
        while(m--) {
            int x, y;
            cin >> x >> y;
            add(x, y); add(y, x); // 这里同一条边是相邻存进“链式前向
星”里的, 一奇一偶
        }
        for(int i = 1; i <= n; i++) {
            if(!dfn[i]) tarjan(i, 0);
        }
        vector<pair<int, int> > ans;
        for(int i = 2; i <= tot; i+=2) {
            if(bridge[i]) {
                int x = ver[i^1], y = ver[i];
                if(x > y) swap(x, y);
                ans.push_back(make_pair(x, y));
            }
        }
    }
}

```

```

cout << ans.size() << '\n';
sort(ans.begin(), ans.end());
for(auto p : ans)
    cout << p.first << ' ' << p.second << '\n';
return 0;
}

```

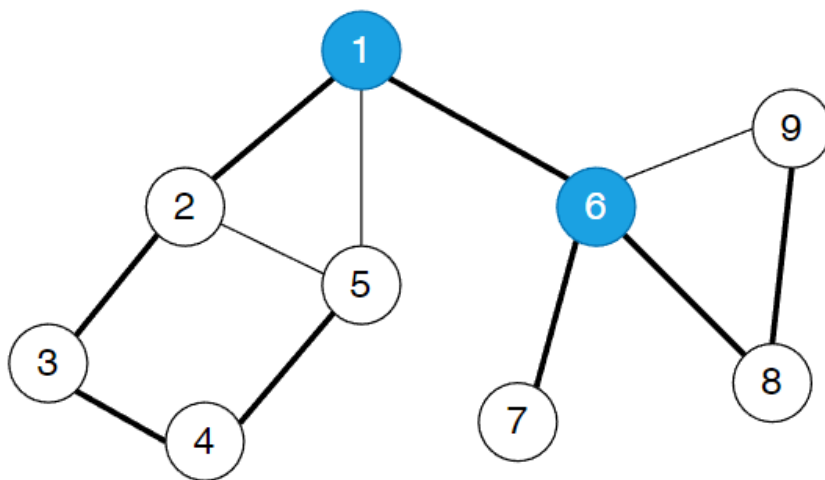
割点判定法则

若 x 不是搜索树的根节点(深度优先遍历的起点), 则 x 是割点当且仅当搜索树上存在 x 的一个子节点 y , 满足:

$$dfn[x] \leq low[y]$$

特别地, 若 x 是搜索树的根节点, 则 x 是割点当且仅当搜索树上存在至少两个子节点 y_1, y_2 满足上述条件。这是因为如果只有一棵子树, 去掉根结点后肯定不会出现多棵子树, 因此就不会是割点。

证明方法与割边的情形类似, 这里就不再赘述(删去 x 后 y 及它的子树上的结点就不能到达 x 的祖先)。在“割边判定法则”画出的例子中, 共有2个割点, 分别是时间戳为 1 和 6 的两个点。



下面的程序求出一张无向图中所有的割点。因为割点判定法则是小于等于号, 所以在求割点时, 不必考虑父节点和重边的问题, 从 x 出发能访问到的所有点的时间戳都可以用来更新 $low[x]$ 。

```

#include <bits/stdc++.h>
using namespace std;

```

```

const int N = 1e5+5;
int head[N], ver[N*2], Next[N*2];
int dfn[N], low[N], n, m, tot = 1, num, root, ans;
bool cut[N];

void add(int x, int y) {
    ver[++tot] = y;
    Next[tot] = head[x];
    head[x] = tot;
}

void tarjan(int x) {
    dfn[x] = low[x] = ++num;
    int flag = 0;
    for(int i = head[x]; i; i = Next[i]) {
        int y = ver[i];
        if(!dfn[y]) {
            tarjan(y);
            low[x] = min(low[x], low[y]);
            if(low[y] >= dfn[x]) {
                flag++;
                if(x != root || flag > 1) cut[x] = true;
            }
        } // 否则，不是搜索树边
        else {
            low[x] = min(low[x], dfn[y]);
        }
    }
}

int main(){
    cin >> n >> m;
    while(m--) {
        int x, y;
        cin >> x >> y;
        if(x == y) continue;
        add(x, y); add(y, x); // 这里同一条边是相邻存进“链式前向
星”里的

```

```
    }  
    for(int i = 1; i <= n; i++) {  
        if(!dfn[i]){  
            root = i;  
            tarjan(i);  
        }  
    }  
  
    for(int i = 1; i <= n; i++) {  
        if(cut[i]) ans++;  
    }  
    cout << ans << '\n';  
    for(int i = 1; i <= n; i++)  
        if(cut[i]) cout << i << ' ';  
    return 0;  
}
```

习题

无向图的割边（桥）

无向图的割点（割顶）